08 — REPORT WRITING GUIDE (Part 2)

Goal: turn your CTF notes into a finished report fast. Part 2 = **12:15–14:15 (2 h)**. If you logged as you went, the report is mostly *assembly + wording*. This sheet maps every technique you used → a finished finding (risk · impact · remediation · verification), and tells you exactly what to type into each box of `Report Template.docx`.

> **Marking reminder:** Pass = easy flag + good report. Dist = +intermediate. HD = +hard. *Every* flag needs: the command(s), a screenshot, the finding, and remediation. One flag written up well beats three flags written up badly.

⛔ Template rules (do NOT lose marks on these)

- **Use the provided template. Do not change its structure, headings, styles, or tables.** You only *fill in* the `[Insert ...]` placeholders and the "Student Entry" cells.
- **Edit in Google Docs:** upload `Report Template.docx` to Drive → open with Google Docs (or `File ▸ Open`). Keep the layout. Export to the required format at the end (`File ▸ Download`).
- **Checkboxes:** to tick a ☐, replace it with ☒ (or ✅). That's *filling in*, not restructuring.
- **Keep the "Guidance" column** in the Executive Summary table — only fill the **Student Entry** column. Don't delete template instruction text unless the box is clearly a placeholder to replace.
- **Don't dump terminal output.** Relevant lines only in the body; long output → **Appendix A**.
- **Label every screenshot** (E-01, E-02 …) and reference it in the text ("see E-03").
- **Redact** passwords/hashes with `••••` *unless the assessment tells you to show them.* Flag values: show them (that's your proof).

🕑 Part-2 battle plan (2 hours)

Time	Do
0:00–0:10	Fill the cover table, Scope, RoE checkboxes. Paste all screenshots into **Appendix A** and give them IDs.
0:10–0:25	**Summary of Findings** table — one row per thing you exploited (F-01…). Pick titles + risk from the Library below.
0:25–1:15	**Detailed Findings** — one block per F-xx. Copy the matching Library card (Description / Impact / Recommendation / Verification), tune to the real host. Reference your E-xx screenshots.
1:15–1:30	**Attack Path Summary** + diagram (your kill-chain as a chain).
1:30–1:45	**Remediation Plan** table (pull straight from each finding's Recommendation + Verification).
1:45–2:00	**Executive Summary** + **Conclusion** (write these LAST, in plain English), then the **Checklist**.

> **Write-as-you-go during Part 1:** for each flag, screenshot **(a)** the command and **(b)** the output/flag, and jot one line: *what it proved + which finding + evidence ID*. That one habit is what makes Part 2 easy.

🧭 The 10 sections — what goes in each (at a glance)

#	Section	Put here	Source
1	Executive Summary	Non-technical: overall risk, main attack path in 1–2 sentences, top fix.	Write last, from §6.
2	Scope & Rules of Engagement	Target IP/host + services; tick the RoE boxes.	nmap; exam brief.
3	Testing Methodology	*Leave the template table as-is.* Optionally name the tools you used per phase.	Template (pre-filled).
4	Summary of Findings	One row per exploited weakness: ID, title, risk, asset, status=Open.	Library below.
5	Detailed Findings	Per finding: info table + Description / Evidence / Impact / Recommendation / Verification.	Library below + your E-xx.
6	Attack Path Summary	Your kill-chain across the 6 rows + a one-line diagram.	Your flag order.
7	Remediation Plan	One row per finding: priority, action, owner, how-to-retest.	Copy from §5 cards.
8	Conclusion	What you proved, which issue mattered most, what to fix first.	Write last.
A	Appendix A — Evidence	Your screenshots/outputs as E-01…, each with a purpose line.	Your loot/ captures.
B	Appendix B — References	OWASP / Microsoft / CVE / MITRE / course refs.	Reference bank below.

🎯 FINDINGS & REMEDIATION LIBRARY (the core — map technique → report)

Find the row(s) for what you actually did. Each card gives you a ready **Finding title**, **Vulnerability type** (for the §5 info table), a typical **Risk**, the **Impact** wording, the **Recommendation** (= §5 Recommendation *and* §7 Remediation action), and the **Verification after fix**. Tune the specifics (host, port, path) to your box.

Foothold / initial access

A. Anonymous SMB share exposes files/credentials — *(smbclient -N, nxc --shares)*

- **Vuln type:** Unauthenticated information disclosure / weak share permissions.
- **Risk:** High (Critical if it leaks credentials or a flag).
- **Impact:** Any unauthenticated attacker on the network can read internal files, notes and credentials from the share, enabling further access without logging in.
- **Recommendation:** Disable anonymous/guest SMB access; require authenticated access; remove sensitive files from shares; apply least-privilege share + NTFS ACLs; disable SMBv1.
- **Verify:** `smbclient -L //HOST -N` and `nxc smb HOST -u '' -p ''` return access denied / no readable shares.

B. Anonymous / world-writable FTP — *(ftp anonymous, put works)*

- **Vuln type:** Anonymous FTP access / insecure write permissions.
- **Risk:** High (Critical if writable + a process executes uploaded files).
- **Impact:** Unauthenticated download of sensitive files; a writable root lets an attacker plant payloads that a scheduled task / web root may execute → remote code execution.
- **Recommendation:** Disable anonymous FTP; if file transfer is needed use SFTP/FTPS with auth; remove write permissions; isolate the FTP root from executable/web paths.
- **Verify:** Anonymous login refused; `put` denied; service offers only authenticated SFTP/FTPS.

C. Web command injection — *(param reaches OS: `;whoami`, `|whoami`, `$(whoami)`)*

- **Vuln type:** OS command injection (OWASP A03).
- **Risk:** Critical.
- **Impact:** Attacker runs arbitrary OS commands as the web service account → full server compromise and a foothold for privilege escalation.
- **Recommendation:** Never pass user input to a shell; use parameterised/native APIs; strict allow-list input validation; run the web service as a low-privileged account; patch the app.
- **Verify:** Injection payloads no longer execute (output not reflected); code review confirms no shell calls on user input.

D. Server-Side Template Injection (SSTI) — *(`{{7*7}}` →49, Jinja2 RCE)*

- **Vuln type:** Server-Side Template Injection → RCE.
- **Risk:** Critical.
- **Impact:** User input evaluated by the template engine lets an attacker execute code on the server, leading to full compromise.
- **Recommendation:** Never render user input as a template; use logic-less/sandboxed templating; context-escape all output; keep the engine patched.
- **Verify:** `{{7*7}}` and equivalents render literally (not evaluated); payloads fail.

E. Unrestricted file upload → web shell — *(upload cmd.php / cmd.aspx)*

- **Vuln type:** Unrestricted file upload / insecure file handling.
- **Risk:** Critical.
- **Impact:** Attacker uploads an executable script and runs commands as the web service → persistent remote code execution.
- **Recommendation:** Allow-list permitted extensions and validate magic bytes/content-type; store uploads outside the web root; randomise filenames; disable script execution in the upload dir.
- **Verify:** Uploading `.php` / `.aspx` / `.jsp` is rejected or stored non-executable; the upload path does not run scripts.

F. Local File Inclusion / path traversal — *(`?file=../../../../windows/win.ini`)*

- **Vuln type:** Path traversal / Local File Inclusion.
- **Risk:** High (Critical if it reaches creds/code or achieves RCE via log poisoning).
- **Impact:** Attacker reads arbitrary files on the server (configs, credentials, flags), aiding further compromise.
- **Recommendation:** Canonicalise and allow-list requested paths; never use user input in file includes; apply least-privilege file permissions to the web account.
- **Verify:** Traversal sequences (`../`, encoded variants) are rejected; only intended files served.

G. Exploitable outdated service (known CVE) — *(searchsploit `<product> <version>`)*

- **Vuln type:** Known vulnerable software version / missing patches.
- **Risk:** Critical (unauth RCE) — else High.
- **Impact:** A public exploit for the running version gives an attacker code execution / initial access without valid credentials.
- **Recommendation:** Patch/upgrade to a fixed version; remove or restrict network exposure of the service; segment and monitor.
- **Verify:** Banner shows the patched version; `searchsploit` shows no matching exploit; PoC fails.
- **Note (exam):** If this was your **one Metasploit use**, say so honestly in the evidence/method — the RoE requires documenting all tools. Manual PoC earns bonus marks.

Credentials / access

H. Weak, guessable or reused password — *(hydra/nxc top-250, season patterns → RDP/WinRM/SSH/SMB)*

- **Vuln type:** Weak authentication / credential guessing.
- **Risk:** High.
- **Impact:** An attacker guesses a valid account and logs in over RDP/WinRM/SMB/SSH, gaining an authenticated foothold; reused passwords extend access to other services.
- **Recommendation:** Enforce a strong password policy and ban common/breached passwords; require MFA on remote access; enable account lockout/throttling; unique credentials per account; disable unused accounts.
- **Verify:** The old password is rejected; the account is not guessable from the top wordlist; lockout/MFA observed on retest.

I. Exposed credentials in files / history / config — *(share notes, `ConsoleHost_history.txt`, `*.config` / `*.xml`, `findstr /si password`, DPAPI/SecureString blob)*

- **Vuln type:** Hard-coded / cleartext credential exposure.
- **Risk:** High → Critical (depends what the creds unlock).
- **Impact:** Credentials recovered from readable files or command history let an attacker authenticate as that user/service and pivot (RDP/WinRM/MSSQL), often to higher privilege.
- **Recommendation:** Remove plaintext secrets from scripts/configs/history; use a secrets manager / managed identities; rotate all exposed credentials; tighten file ACLs; clear PowerShell history.
- **Verify:** No secrets found via `findstr`/grep sweeps; history cleared; rotated creds no longer valid; file ACLs restrict access.

Privilege escalation (Windows)

J. ⭐ SeImpersonate → Potato → SYSTEM — *(`whoami /priv` , SigmaPotato/PrintSpoofer/GodPotato)*

- **Vuln type:** Excessive privilege (SeImpersonate/SeAssignPrimaryToken) → token-impersonation privesc.
- **Risk:** High (part of a Critical end-to-end chain).
- **Impact:** A low-privileged service context abuses impersonation to execute code as **NT AUTHORITY\SYSTEM**, fully compromising the host.
- **Recommendation:** Remove `SeImpersonatePrivilege` / `SeAssignPrimaryToken` from accounts that don't need it; run services under least-privilege virtual/managed accounts; keep Windows patched (DCOM/RPC mitigations); monitor for token-impersonation behaviour.
- **Verify:** `whoami /priv` shows the privilege removed/disabled for that account; the Potato technique no longer yields SYSTEM.

K. Writable scheduled-task / service script (token-context hijack) — *(schtasks, icacls shows Users:(W) on a SYSTEM task's script)*

- **Vuln type:** Insecure file permissions on a privileged task/service target.
- **Risk:** High.
- **Impact:** A low-privileged user overwrites a script/binary that a SYSTEM/admin task runs, so their code executes in that privileged context → privilege escalation.
- **Recommendation:** Correct ACLs so non-privileged users cannot modify task/service targets; store scripts in protected directories; run tasks with least privilege; sign and integrity-check scripts.
- **Verify:** `icacls` shows no write for `Users` /your account on the target; overwrite is denied.

L. Writable service binary / insecure service ACL — *(`sc qc` , `icacls service.exe` → (M)/(W)/(F))*

- **Vuln type:** Weak service permissions (writable binary or reconfigurable service).
- **Risk:** High.
- **Impact:** An attacker replaces the service executable (or reconfigures its binPath) so their payload runs as the service account (often SYSTEM) on restart → privilege escalation.
- **Recommendation:** Restrict write access on service binaries and their folders; lock down service configuration rights (`sc sdset`); run services as least-privilege accounts.
- **Verify:** `icacls` denies write to non-admins; the binary cannot be replaced; `sc config` denied.

M. Unquoted service path — *(`wmic service get name,pathname,startmode` → space in unquoted path)*

- **Vuln type:** Unquoted service path.
- **Risk:** High (needs a writable intermediate dir).
- **Impact:** Windows may execute an attacker-planted binary (e.g. `C:\Program.exe`) with the service's privileges.
- **Recommendation:** Quote all service `binPath` values; remove write access on the path's parent directories.
- **Verify:** `wmic` / `sc qc` shows the path fully quoted; no writable intermediate directories.

N. AlwaysInstallElevated — *(both HKLM+HKCU `AlwaysInstallElevated = 0x1`)*

- **Vuln type:** Dangerous installer policy (AlwaysInstallElevated).
- **Risk:** High.
- **Impact:** Any user can install an MSI that runs as SYSTEM, giving a direct privilege-escalation path.
- **Recommendation:** Disable the `AlwaysInstallElevated` policy in both HKLM and HKCU (set to 0 / remove via GPO).
- **Verify:** `reg query ... /v AlwaysInstallElevated` returns `0x0` or absent in both hives; a test MSI no longer installs elevated.

O. MSSQL `xp_cmdshell` command execution — *(sqsh/mssqlclient → `EXEC xp_cmdshell`)*

- **Vuln type:** Database OS command execution via `xp_cmdshell` .
- **Risk:** Critical (DB creds → OS as the SQL service account, usually with SeImpersonate → SYSTEM).
- **Impact:** An attacker with SQL access runs OS commands as the SQL service account, then escalates to SYSTEM — full host compromise from a database login.
- **Recommendation:** Disable `xp_cmdshell` ; run SQL Server under a least-privilege account without SeImpersonate; enforce strong/Windows authentication; patch; restrict network access to the DB.
- **Verify:** `sp_configure 'xp_cmdshell'` = 0 and `EXEC xp_cmdshell` is denied; SQL service account lacks impersonation rights.

📊 Risk rating quick-picker

Pick the single rating that best fits. When in doubt, rate by **impact × ease**.

Rating	Use when
Critical	Unauthenticated RCE, or any chain that reaches **SYSTEM/Administrator**, or exposed creds that unlock admin. (Your overall box rating if you got SYSTEM.)
High	Authenticated RCE, privilege escalation needing a foothold, sensitive data/credential exposure, weak remote-login passwords.
Medium	Info disclosure without direct compromise, LFI to non-sensitive files, misconfig that needs chaining.
Low	Minor info leak, verbose banners/headers, best-practice deviations with little direct impact.

Overall Risk (Exec Summary) = the highest single finding, usually **Critical** if you reached SYSTEM. **Primary Root Cause** = the one theme (misconfiguration / weak access control / exposed credentials / missing patches). **Top Remediation Priority** = fix the step that broke the chain earliest (e.g. remove the exposed creds, or patch the exploited service).

🧩 Attack Path Summary — fill the 6 rows + diagram

Map your real kill-chain onto the template rows (example = a common Windows path):

Template row	Example entry (replace with yours)
Initial access / foothold	Anonymous SMB share read → easy flag + notes (E-01).
Local enumeration	`whoami /priv` showed SeImpersonatePrivilege (E-04).
Credential or config weakness	Password found in `ConsoleHost_history.txt` (E-05).
Service or application abuse	MSSQL login → `xp_cmdshell` as svc account (E-06).
Privilege escalation proof	SigmaPotato → `whoami` = `nt authority\system` (E-07).
Final impact	Read `flag_hard.txt` from Administrator Desktop (E-08).

Diagram (one line): `Anon SMB → Exposed Creds → MSSQL xp_cmdshell → SeImpersonate → SigmaPotato → SYSTEM → all flags`

🖱️ Appendix A — evidence discipline

- One screenshot = one **E-ID**. Caption = *what command* + *what it proves*.
- Each Detailed Finding references its evidence ("Evidence: E-03, E-04").
- Keep each screenshot tight: the command + the few relevant output lines (+ the flag).
- Capture during Part 1 to `loot/` (the cheatsheets' `ev` / `script` wrappers already tee output).

Evidence ID	Command / Source	Purpose
E-01	`smbclient //HOST/Share -N`	Anonymous share access + easy flag.
E-02	`nmap -sV -p- HOST`	Open ports / service versions (scope).
E-03	web exploit request/response	Proof of the foothold vector.
E-04	`whoami /priv`	Shows SeImpersonate (privesc enabler).
E-05	creds source (history/config)	Where the credential came from.
E-06	`EXEC xp_cmdshell 'whoami'`	Command exec as service account.
E-07	`SigmaPotato ... whoami`	`nt authority\system` — privesc proof.
E-08	`type ...\flag_hard.txt`	Final flag captured.

📚 Appendix B — reference bank (ready to paste)

Use a consistent style. Pick the ones relevant to your findings:

- **OWASP Top 10 (2021):** A01 Broken Access Control; A03 Injection; A05 Security Misconfiguration; A07 Identification & Authentication Failures. — https://owasp.org/Top10/
- **OWASP Testing Guide / Cheat Sheets** (command injection, file upload, SSTI, path traversal). — https://cheatsheetseries.owasp.org/
- **MITRE ATT&CK:** Valid Accounts (T1078); Command & Scripting Interpreter (T1059); Access Token Manipulation (T1134); Exploitation for Privilege Escalation (T1068); Unsecured Credentials (T1552). — https://attack.mitre.org/
- **Microsoft — SMB security / disable anonymous access;** Service account least privilege; AlwaysInstallElevated (disable); `xp_cmdshell` Server Configuration Option. — https://learn.microsoft.com/
- **CIS Microsoft Windows Benchmarks** (hardening baselines). — https://www.cisecurity.org/benchmark/
- **CVE / NVD:** cite the specific CVE for any versioned service you exploited. — https://nvd.nist.gov/
- **Course materials:** ISEC3002 lectures & workshop activities (Tut 3/4/7/8/9).

✍️ Worked micro-examples (copy the shape, change the facts)

Finding F-01 — Anonymous SMB Share Exposes Sensitive Files (easy flag)

- Risk: **High** · Asset: `HOST (445/tcp, SMB)` · Type: Unauthenticated info disclosure · Evidence: E-01 · Impact: unauth file read · Priority: **Immediate**.
- *Description:* The `Public` share allowed anonymous (null-session) access and contained internal notes and `flag_easy.txt`, readable without credentials.
- *Evidence:* `smbclient //HOST/Public -N` listed and downloaded the files (E-01).
- *Impact:* Any unauthenticated attacker on the network can read internal documents and credentials, providing a foothold.
- *Recommendation:* Disable anonymous SMB; require authentication; least-privilege share/NTFS ACLs; remove sensitive files from the share.
- *Verification:* `smbclient -L //HOST -N` returns access denied; no share is anonymously readable.

Finding F-03 — Privilege Escalation to SYSTEM via SeImpersonate (hard flag)

- Risk: **High** (chain = Critical) · Asset: `HOST (Windows)` · Type: Excessive privilege / token impersonation · Evidence: E-04, E-07 · Priority: **Immediate**.
- *Description:* The service account held `SeImpersonatePrivilege`. Using a token-impersonation tool (SigmaPotato) this was abused to execute code as `NT AUTHORITY\SYSTEM`.
- *Evidence:* `whoami /priv` (E-04) shows SeImpersonate; SigmaPotato `whoami` returns `nt authority\system` (E-07); `flag_hard.txt` read thereafter (E-08).
- *Impact:* Complete compromise of the host — full control of files, services and accounts.
- *Recommendation:* Remove SeImpersonate from accounts that don't need it; least-privilege service accounts; keep the OS patched; monitor for token impersonation.
- *Verification:* `whoami /priv` no longer lists the privilege for that account; the technique fails.

✅ Final checklist (mirror of the template's)

- ☒ Executive summary written for a non-technical reader.

- ☒ Each finding has risk, evidence, impact, remediation.
- ☒ Screenshots labelled (E-xx) and referenced in the text.
- ☒ Only authorised testing activity included.
- ☒ Credentials redacted unless required by the assessment.
- ☒ Report is professional, concise, no unnecessary terminal history (long output → Appendix A).
- ☒ Template structure unchanged; exported in the required format from Google Docs.

> One well-documented flag = Pass. Keep the chain honest, the evidence tight, and the remediation specific to the misconfiguration you actually found.